

Доклад

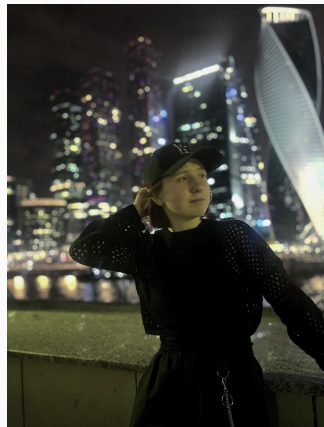
Пакетный фильтр на базе ОС Linux

Верниковская Е. А., НПИбд-01-23

28 марта 2025

Российский университет дружбы народов, Москва, Россия

- Верниковская Екатерина Андреевна
- Студентка
- Российский университет дружбы народов
- 1132236136@pfur.ru



Актуальность темы и проблема:

сетевые атаки и угрозы постоянно эволюционируют, поэтому надежная фильтрация трафика необходима для защиты серверов и сетей. Пакетные фильтры в Linux обеспечивают эффективную защиту и контроль доступа. Без настройки фильтрации система уязвима для несанкционированного доступа, DDoS-атак и вредоносного трафика. Поэтому важно правильно настроить и оптимизировать фильтры для повышения безопасности без ущерба для производительности

Объект и предмет

исследования:

пакетный фильтр на
базе ОС Linux

Цель:

цель данного доклада - рассмотреть принципы работы пакетных фильтров в Linux, их роль в обеспечении сетевой безопасности, а также сравнить основные инструменты (iptables, nftables, eBPF) и продемонстрировать примеры их настройки

Задачи

исследования:

изучить пакетные фильтры на базе операционной системы Linux

Материалы и

методы и

инструменты

исследования:

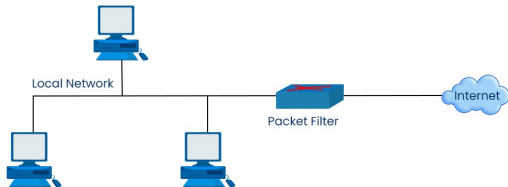
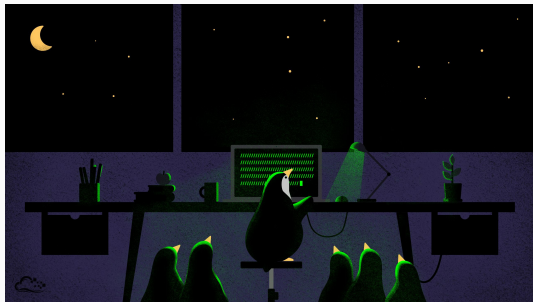
интернет-ресурсы, аналитика и практические навыки работы на своей операционной системе Linux (Ubuntu)

Введение

Атаки и киберугрозы представляют собой серьезную проблему для информационных технологий и безопасности данных в современном мире. С каждым годом количество кибератак возрастает, становясь все более изощренными и разнообразными. Киберугрозы могут варьироваться от вирусов и троянов до сложных атак, таких как DDoS (Distributed Denial of Service) и phishing (фишинг). В ответ на эти угрозы, многие организации и пользователи обращаются к эффективным методам защиты, одним из которых является пакетный фильтр.



Что такое пакетный фильтр и зачем он нужен?



Пакетный фильтр (Packet Filter - PF) — это элемент брандмауэра, позволяющий управлять прохождением трафика по указанным протоколам, разрешая или запрещая передачу пакетов, удовлетворяющих заданным условиям. Простыми словами это программа которая просматривает заголовки пакетов по мере их прихода, и решает дальнейшую судьбу всего пакета.

Основные компоненты пакетного фильтра

1. Сетевые пакеты и их заголовки

Каждый сетевой пакет содержит заголовок, который несет важную информацию о передаваемых данных.

Типы заголовков:

- Заголовок Ethernet (Канальный уровень, L2)
- Заголовок IP (Сетевой уровень, L3)
- Заголовки TCP и UDP (Транспортный уровень, L4)

2. Сетевые интерфейсы

Сетевые интерфейсы — это точки подключения к сети, через которые происходит обмен данными. Пакетные фильтры работают непосредственно с сетевыми интерфейсами, перехватывая входящие и исходящие пакеты, которые проходят через них. В Linux это может быть интерфейс Ethernet, Wi-Fi и другие.

Основные компоненты пакетного фильтра

3. Правила фильтрации

Правила фильтрации представляют собой набор условий, которые определяют, как обрабатываются пакеты. Типы правил:

- Разрешающие (ALLOW) — пакеты разрешаются к передаче
- Блокирующие (DENY) — пакеты отклоняются
- Журналирующие (LOG) — информация о пакетах записывается в журнал

4. Таблицы

Правила, используемые пакетным фильтром, группируются в таблицы, каждая из которых предназначена для определённой задачи. Основные таблицы:

- filter
- nat
- mangle
- raw

Основные компоненты пакетного фильтра

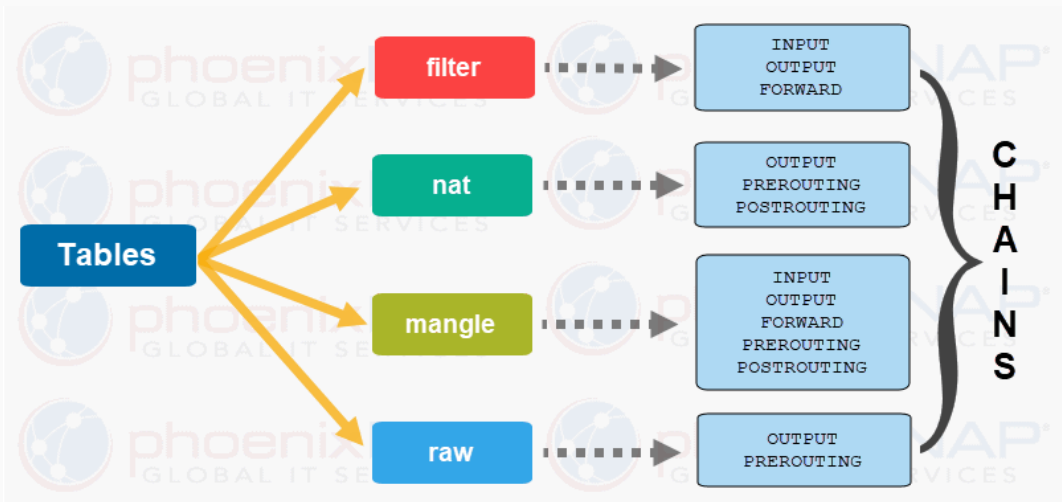


Рис. 1: Таблицы и цепочки

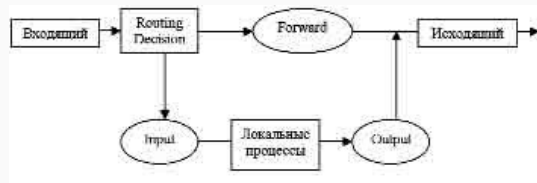
Основные компоненты пакетного фильтра

5. Цепочки (chains)

Цепочки — это последовательности правил внутри таблицы фильтрации.

Основные цепочки:

- INPUT
- OUTPUT
- FORWARD
- PREROUTING
- POSTROUTING



6. Программное обеспечение для фильтрации

В Linux для реализации пакетной фильтрации используется специализированное программное обеспечение. Примеры:

- iptables - классический инструмент для настройки фильтрации на уровне IP
- nftables - новая, более современная система фильтрации
- eBPF - код на уровне ядра

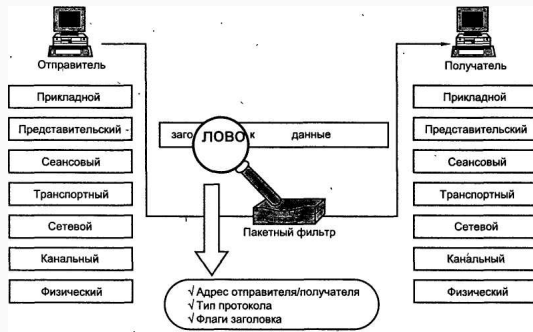
7. Журналирование и мониторинг

Процесс записи событий и действий пакетного фильтра в журнал.

Журналирование позволяет администраторам отслеживать подозрительную активность и анализировать успешные и неуспешные попытки доступа. Это важно для дальнейшего реагирования на инциденты и улучшения мер безопасности.

Принцип работы пакетного фильтра

1. Перехват пакетов
2. Анализ заголовков пакетов
3. Применение правил фильтрации
4. Принятие решения и передача пакета



Встроенные механизмы фильтрации в Linux

Linux использует мощную подсистему Netfilter, встроенную в ядро, которая отвечает за обработку сетевых пакетов.



Рис. 3: Межсетевой экран

Для взаимодействия с Netfilter в Linux используются несколько инструментов:

- iptables – традиционный инструмент управления фильтрацией пакетов.
- nftables – современная альтернатива, более мощная и гибкая.
- eBPF – новый механизм, обеспечивающий высокоэффективную обработку пакетов на уровне ядра.

Обзор iptables, nftables и eBPF

iptables:

Это один из самых популярных инструментов для создания правил фильтрации на основе пакетов. Он существует с 1998 года и является основным средством для настройки брандмауэров в Linux.

nftables:

Это современная замена iptables, представленная в ядре Linux 3.13

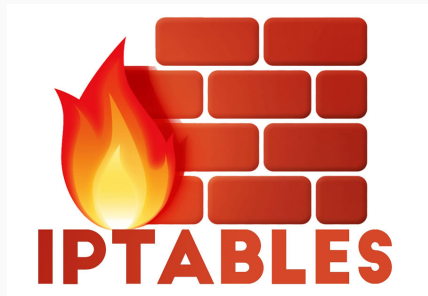


eBPF (extended Berkeley Packet Filter):

Это мощный механизм, позволяющий запуска кода непосредственно в ядре Linux для фильтрации пакетов и мониторинга

Работа iptables

Операционные системы Linux, на которых чаще всего и функционируют серверы (виртуальные машины), имеют встроенный инструмент защиты – программный фильтр **iptables**. Он работает путем проверки пакетов данных на соответствие определенным критериям и выполнения заданных действий, если пакеты соответствуют этим критериям. Эти критерии и действия определяются в таблицах, которые состоят из набора правил.



Основные таблицы и цепочки iptables

Четыре основные таблицы iptables:

1. Filter - это основная таблица, используемая для фильтрации пакетов
2. NAT - эта таблица используется для настройки NAT
3. Mangle - эта таблица используется для специальной обработки пакетов
4. Raw - эта таблица используется для обхода системы отслеживания состояний

Каждая таблица состоит из набора цепочек:

1. INPUT - эта цепочка применяется к пакетам, которые предназначены для самой системы
2. FORWARD - эта цепочка применяется к пакетам, которые проходят через систему
3. OUTPUT - эта цепочка применяется к пакетам, которые исходят из системы

Правила фильтрации и их синтаксис в iptables

Общий синтаксис запуска программы выглядит следующим образом: *iptables -t таблица действие цепочка дополнительные_параметры*

Перечень основных действий:

В качестве дополнительных

параметров используются опции:

1. -D – удалить текущее правило
2. -L – вывести правила текущей цепочки
3. -S – вывести все активные правила
4. -F – очистить все правила
5. -N – создать цепочку
6. -X – удалить цепочку
7. -P – установить действие «по умолчанию»

1. -p – вручную установить протокол
2. -s – указать статичный IP-адрес оборудования, откуда отправляется пакет данных
3. -d – установить IP получателя
4. -i – настроить входной сетевой интерфейс
5. -j – выбрать действие при подтверждении правила



nftables — это современный механизм фильтрации пакетов в Linux, который заменяет устаревший iptables. Он обеспечивает более простую и удобную работу с правилами, а также улучшенную производительность.

Основные таблицы и цепочки nftables

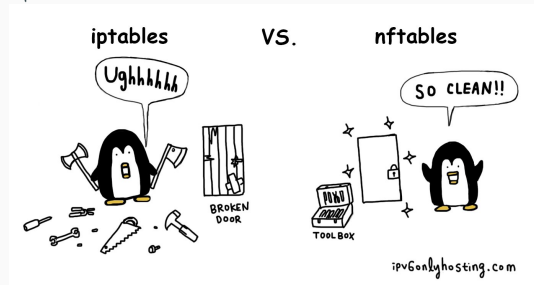
В отличие от iptables в nftables вместо разделения на отдельные таблицы (filter, nat, mangle, raw), nftables предлагает более общую структуру, где вы сами определяете таблицы и цепочки, адаптированные к вашим конкретным потребностям.

Таблицы определяются с указанием семейства адресов. Таблицы могут относиться к одному из пяти семейств:

- ip: (т.е. IPv4) — семейство по умолчанию; используется, если семейство не было указано
- ip6: IPv6
- inet: IPv4 и IPv6 (используется чаще всего, делает правила более универсальными)
- arp: ARP
- bridge: Мосты

Основные таблицы и цепочки nftables

В отличие от iptables, в nftables отсутствуют также и встроенные цепочки.



Есть два типа цепочек:

- **Base chains (базовые цепочки)** – напрямую обрабатывают пакеты из сетевого стека (input, output, forward и т. д.)
- **Regular chains (обычные цепочки)** – служат вспомогательными и вызываются из других цепочек

Правила фильтрации и их синтаксис в nftables

Синтаксис nftables разработан для большей ясности и удобства по сравнению с iptables: *nft* “команда” “объект” “путь к объекту” “параметры”

- Команда — add, insert, delete, replace, rename, list, flush...
- Объект — table, chain, rule, set, ruleset...
- Путь к объекту зависит от типа. Например, у таблицы это “семейство” “название”. У правила — гораздо длиннее: “семейство таблицы” “название таблицы” “название цепочки”
- Параметры зависят от типа объекта. Для правила это условие отбора пакетов и действие, применяемое к отобранным пакетам



eBPF (англ. extended Berkley Packet Filter — расширенный фильтр пакетов Беркли) — это мощный механизм в ядре Linux, который позволяет безопасно выполнять пользовательский код на уровне ядра без необходимости его модификации. Благодаря eBPF можно анализировать и фильтровать сетевой трафик с высокой производительностью и низкими затратами на системные ресурсы. В отличие от классических инструментов (iptables, nftables), eBPF обладает высокой производительностью и гибкостью, что делает его особенно полезным для современных задач сетевой безопасности.

Работа eBPF

eBPF работает следующим образом:

1. Пользователь загружает eBPF-программу в ядро
2. Ядро проверяет программу на безопасность
3. Программа подключается к одной из точек (hook points) ядра
4. При прохождении сетевого пакета eBPF-программа анализирует его заголовки и принимает решение
5. Результат обработки применяется к пакету в реальном времени

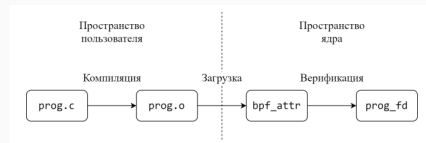


Рис. 4: Жизненный цикл eBPF программы

Основные точки внедрения eBPF для фильтрации

1. XDP (eXpress Data Path)

XDP обрабатывает пакеты на уровне сетевого драйвера, еще до попадания в стек ядра.

2. TC (Traffic Control) BPF

TC eBPF работает на уровне сетевого стека ядра и позволяет фильтровать и изменять пакеты после их принятия системой.

3. Socket Filtering (SOCK_FILTER)

Фильтрует трафик на уровне сокетов, прежде чем он попадет в пользовательские приложения.

Практическое применение. Пример работы с iptables

Перед активацией
приложение требуется
установить из
стандартного
репозитория Linux.
Так, для инсталляции в
Ubuntu подойдет
команда: *sudo apt
install iptables*

```
eavernikovskaya@ubuntu-katerok:~$ sudo apt install iptables
Чтение списков пакетов... Готово
Построение дерева зависимостей... Готово
Чтение информации о состоянии... Готово
Уже установлен пакет iptables самой новой версии (1.8.10-3ubuntu2).
iptables помечен как установленный вручную.
Обновлено 0 пакетов, установлено 0 новых пакетов, для удаления отмечено 0 пакетов, и 269 пакетов не обновлено.
eavernikovskaya@ubuntu-katerok:~$
```

Рис. 5: Установка iptables

Практическое применение. Пример работы с iptables

```
eavernikovskaya@ubuntu-katerok:~$ sudo iptables -L INPUT
Chain INPUT (policy ACCEPT)
target     prot opt source                destination
eavernikovskaya@ubuntu-katerok:~$ sudo iptables -L FORWARD
Chain FORWARD (policy ACCEPT)
target     prot opt source                destination
eavernikovskaya@ubuntu-katerok:~$ sudo iptables -L OUTPUT
Chain OUTPUT (policy ACCEPT)
target     prot opt source                destination
eavernikovskaya@ubuntu-katerok:~$
```

Рис. 6: Цепочки правил

Посмотрим конкретные цепочки правил. Команда чтобы вывести на экран конкретную цепочку правил выглядит следующим образом: *sudo iptables -L [имя цепочки]*

Практическое применение. Пример работы с iptables

Зададим правила «по умолчанию» вручную: `sudo iptables -p INPUT ACCEPT`, `sudo iptables -p OUTPUT ACCEPT` и `sudo iptables -p FORWARD DROP`. Здесь мы разрешаем все цепочки INPUT и OUTPUT, запрещаем FORWARD. А после проверим командой `iptables -L -n -v` правильность заданных правил

```
eavernikovskaya@ubuntu-katerok:~$ sudo iptables -P INPUT ACCEPT
eavernikovskaya@ubuntu-katerok:~$ sudo iptables -P OUTPUT ACCEPT
eavernikovskaya@ubuntu-katerok:~$ sudo iptables -P FORWARD DROP
eavernikovskaya@ubuntu-katerok:~$
```

Рис. 7: Правила по-умолчанию

```
eavernikovskaya@ubuntu-katerok:~$ sudo iptables -L -n -v
Chain INPUT (policy ACCEPT 1165 packets, 250K bytes)
 pkts bytes target    prot opt in     out     source                   destination

Chain FORWARD (policy DROP 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source                   destination

Chain OUTPUT (policy ACCEPT 555 packets, 99023 bytes)
 pkts bytes target    prot opt in     out     source                   destination
eavernikovskaya@ubuntu-katerok:~$
```

Рис. 8: Список правил

Практическое применение. Пример работы с iptables

```
eavernikovskayaubuntu-katerok:~$ sudo iptables -t filter -L -n -v
Chain INPUT (policy ACCEPT 5766 packets, 2347K bytes)
pkts bytes target      prot opt in     out    source            destination

Chain FORWARD (policy DROP 0 packets, 0 bytes)
pkts bytes target      prot opt in     out    source            destination

Chain OUTPUT (policy ACCEPT 2992 packets, 543K bytes)
pkts bytes target      prot opt in     out    source            destination
eavernikovskayaubuntu-katerok:~$ sudo iptables -t nat -L -n -v
Chain PREROUTING (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target      prot opt in     out    source            destination

Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target      prot opt in     out    source            destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target      prot opt in     out    source            destination

Chain POSTROUTING (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target      prot opt in     out    source            destination
eavernikovskayaubuntu-katerok:~$ sudo iptables -t mangle -L -n -v
Chain PREROUTING (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target      prot opt in     out    source            destination

Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target      prot opt in     out    source            destination

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target      prot opt in     out    source            destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target      prot opt in     out    source            destination

Chain POSTROUTING (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target      prot opt in     out    source            destination
eavernikovskayaubuntu-katerok:~$ sudo iptables -t raw -L -n -v
Chain PREROUTING (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target      prot opt in     out    source            destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target      prot opt in     out    source            destination
eavernikovskayaubuntu-katerok:~$
```

По умолчанию iptables работает с таблицей filter. Для просмотра правил в этой таблице используем: *sudo iptables -t filter -L -n -v*.

Аналогично посмотрим правила в таблицах nat, mangle и raw: *sudo iptables -t nat -L -n -v*, *sudo iptables -t mangle -L -n -v* и *sudo iptables -t raw -L -n -v*

Рис. 9: Просмотр правил в конкретных таблицах

Практическое применение. Пример работы с iptables

Создадим
пользовательскую
цепочку в таблице
фильтрации: *sudo*
iptables -N MYCHAIN

```
eavernikovskaya@ubuntu-katerok:~$ sudo iptables -N MYCHAIN
eavernikovskaya@ubuntu-katerok:~$ sudo iptables -t filter -L -n -v
Chain INPUT (policy ACCEPT 6249 packets, 2446K bytes)
 pkts bytes target    prot opt in     out     source                   destination
Chain FORWARD (policy DROP 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source                   destination
Chain OUTPUT (policy ACCEPT 3144 packets, 576K bytes)
 pkts bytes target    prot opt in     out     source                   destination
Chain MYCHAIN (0 references)
 pkts bytes target    prot opt in     out     source                   destination
eavernikovskaya@ubuntu-katerok:~$
```

Рис. 10: Создание цепочки с помощью iptables

Практическое применение. Пример работы с iptables

```
eavernikovskaya@ubuntu-katerok:~$ sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT
eavernikovskaya@ubuntu-katerok:~$
```

Рис. 11: Добавление правила в стандартную цепочку с помощью iptables

```
eavernikovskaya@ubuntu-katerok:~$ sudo iptables -A MYCHAIN -s 192.168.1.0/24 -j DROP
eavernikovskaya@ubuntu-katerok:~$
```

Рис. 12: Добавление правил в созданную цепочку с помощью iptables

Добавим правила в стандартную цепочку (например, разрешение ssh):

sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT

Также добавим правила в созданную цепочку (например, заблокируем IP адрес 192.168.1.0/24): *sudo iptables -A MYCHAIN -s 192.168.1.0/24 -j DROP*

Практическое применение. Пример работы с iptables

И проверим
правильность
выполнения

```
eavernikovskaya@ubuntu-katerok:~$ sudo iptables -t filter -L -n -v
Chain INPUT (policy ACCEPT 7031 packets, 2614K bytes)
 pkts bytes target    prot opt in     out     source            destination
    0    0 ACCEPT    6     --  *      *        0.0.0.0/0         0.0.0.0/0         tcp dpt:22

Chain FORWARD (policy DROP 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source            destination

Chain OUTPUT (policy ACCEPT 3295 packets, 600K bytes)
 pkts bytes target    prot opt in     out     source            destination

Chain MYCHAIN (0 references)
 pkts bytes target    prot opt in     out     source            destination
    0    0 DROP     0     --  *      *       192.168.1.0/24    0.0.0.0/0
```

Рис. 13: Просмотр правил в таблице фильтрации

Практическое применение. Пример работы с iptables

```
eavernikovskaya@ubuntu-katerok:~$ sudo iptables -A INPUT -j MYCHAIN
eavernikovskaya@ubuntu-katerok:~$ sudo iptables -L
Chain INPUT (policy ACCEPT)
target     prot opt source                destination            tcp dpt:ssh
ACCEPT     tcp  --  anywhere              anywhere
MYCHAIN    all  --  anywhere              anywhere

Chain FORWARD (policy DROP)
target     prot opt source                destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source                destination

Chain MYCHAIN (1 references)
target     prot opt source                destination
DROP      all  --  192.168.1.0/24        anywhere
eavernikovskaya@ubuntu-katerok:~$
```

Теперь можно использовать созданную цепочку. Например, разрешить доступ по цепочке MYCHAIN: *sudo iptables -A INPUT -j MYCHAIN*

Рис. 14: Использование созданной цепочки

Практическое применение. Пример работы с iptables

Далее для сохранения текущих правил iptables используем:

sudo iptables-save | sudo tee /etc/iptables/rules.v4

Чтобы настройки

применялись после

перезагрузки используем

команду *sudo iptables-restore < /etc/iptables/rules.v4*

```
eavernikovskaya@ubuntu-katerok:~$ sudo iptables-save | sudo tee /etc/iptables/rules.v4
# Generated by iptables-save v1.8.10 (nf_tables) on Sun Mar 16 16:00:30 2025
*filter
:INPUT ACCEPT [7334:2673144]
:FORWARD DROP [0:0]
:OUTPUT ACCEPT [3326:602341]
:MYCHAIN - [0:0]
-A INPUT -p tcp -m tcp --dport 22 -j ACCEPT
-A INPUT -j MYCHAIN
-A MYCHAIN -s 192.168.1.0/24 -j DROP
COMMIT
# Completed on Sun Mar 16 16:00:30 2025
eavernikovskaya@ubuntu-katerok:~$
```

Рис. 15: Сохранение конфигураций (iptables)

```
eavernikovskaya@ubuntu-katerok:~$ sudo iptables-restore < /etc/iptables/rules.v4
eavernikovskaya@ubuntu-katerok:~$
```

Рис. 16: Восстановление конфигураций (iptables)

Практическое применение. Пример работы с iptables

```
eavernikovskaya@ubuntu-katerok:~$ reboot
```

Рис. 17: Перезапуск системы

```
eavernikovskaya@ubuntu-katerok:~$ sudo iptables -L
[sudo] password for eavernikovskaya:
Chain INPUT (policy ACCEPT)
target     prot opt source                destination          tcp dpt:ssh
ACCEPT    tcp  --  anywhere              anywhere
MYCHAIN    all  --  anywhere              anywhere

Chain FORWARD (policy DROP)
target     prot opt source                destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source                destination

Chain MYCHAIN (1 references)
target     prot opt source                destination
DROP      all  --  192.168.1.0/24        anywhere
eavernikovskaya@ubuntu-katerok:~$
```

Далее перезапустим систему и проверим остались ли наши ранее заданные правила

Рис. 18: Проверка правил (iptables)

Практическое применение. Пример работы с iptables

Если требуется полное удаление правил, применяется команда: *sudo iptables -F*

```
eavernikovskaya@ubuntu-katerok:~$ sudo iptables -F
eavernikovskaya@ubuntu-katerok:~$ sudo iptables -L
Chain INPUT (policy ACCEPT)
target     prot opt source                destination

Chain FORWARD (policy DROP)
target     prot opt source                destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source                destination

Chain MYCHAIN (0 references)
target     prot opt source                destination
```

Рис. 19: Полное удаление правил

Далее удалим созданную нами ранее цепочку с помощью утилиты *sudo iptables -X*

```
eavernikovskaya@ubuntu-katerok:~$ sudo iptables -X MYCHAIN
eavernikovskaya@ubuntu-katerok:~$ sudo iptables -L
Chain INPUT (policy ACCEPT)
target     prot opt source                destination

Chain FORWARD (policy DROP)
target     prot opt source                destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source                destination
eavernikovskaya@ubuntu-katerok:~$
```

Рис. 20: Удаление цепочки

Практическое применение. Пример работы с iptables

Снова сохраним текущие правила и применим iptables-restore

```
eavernikovskaya@ubuntu-katerok:~$ sudo iptables-save | sudo tee /etc/iptables/rules.v4
# Generated by iptables-save v1.8.10 (nf_tables) on Sun Mar 16 16:06:12 2025
*filter
:INPUT ACCEPT [7497:3267635]
:FORWARD DROP [0:0]
:OUTPUT ACCEPT [5439:2484567]
COMMIT
# Completed on Sun Mar 16 16:06:12 2025
eavernikovskaya@ubuntu-katerok:~$
```

Рис. 21: Сохранение конфигураций (iptables) (2)

```
eavernikovskaya@ubuntu-katerok:~$ sudo iptables-restore < /etc/iptables/rules.v4
eavernikovskaya@ubuntu-katerok:~$
```

Рис. 22: Восстановление конфигураций (iptables) (2)

```
eavernikovskaya@ubuntu-katerok:~$ reboot
```

Рис. 23: Перезапуск системы

```
eavernikovskaya@ubuntu-katerok:~$ sudo iptables -L
[sudo] password for eavernikovskaya:
Chain INPUT (policy ACCEPT)
target     prot opt source                destination

Chain FORWARD (policy DROP)
target     prot opt source                destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source                destination
eavernikovskaya@ubuntu-katerok:~$
```

Рис. 24: Проверка правил (iptables) (2)

Практическое применение. Пример работы с iptables

Выполним команду *ping*. Ответы от сервера показывают, что он доступен. Далее изменим политику по умолчанию для цепочки OUTPUT, устанавливая ее в DROP. Это означает, что все исходящие пакеты будут сбрасываться. После выполнения команды, запросы не смогут покинуть систему. При попытке отправить ping на google.com возникает ошибка. Это связано с тем, что все исходящие пакеты, включая DNS-запросы, блокируются установленными правилами iptables.

```
eavernikovskaya@ubuntu-katerok:~$ ping google.com
PING google.com (142.250.74.206): 56(84) bytes of data:
64 bytes from fra24s02-ln-f14.1e100.net (142.250.74.206): icmp_seq=1 ttl=59 time=44.5 ms
64 bytes from fra24s02-ln-f14.1e100.net (142.250.74.206): icmp_seq=2 ttl=59 time=44.9 ms
64 bytes from fra24s02-ln-f14.1e100.net (142.250.74.206): icmp_seq=3 ttl=59 time=43.5 ms
64 bytes from fra24s02-ln-f14.1e100.net (142.250.74.206): icmp_seq=4 ttl=59 time=44.2 ms
64 bytes from fra24s02-ln-f14.1e100.net (142.250.74.206): icmp_seq=5 ttl=59 time=48.5 ms
^Z
[1]+  Stopped                  ping google.com
eavernikovskaya@ubuntu-katerok:~$ sudo iptables -P OUTPUT DROP
[sudo] password for eavernikovskaya:
eavernikovskaya@ubuntu-katerok:~$ sudo iptables -L -n -v
Chain INPUT (policy ACCEPT 815K packets, 1004M bytes)
 pkts bytes target     prot opt in     out     source                   destination

Chain FORWARD (policy DROP 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source                   destination

Chain OUTPUT (policy DROP 69 packets, 14007 bytes)
 pkts bytes target     prot opt in     out     source                   destination
eavernikovskaya@ubuntu-katerok:~$ ping google.com
ping: google.com: Temporary failure in name resolution
eavernikovskaya@ubuntu-katerok:~$
```

Рис. 25: Пример

Практическое применение. Пример работы с nftables

```
eavernikovskaya@ubuntu-katerok:~$ sudo apt-get install nftables
[sudo] password for eavernikovskaya:
Чтение списков пакетов... Готово
Построение дерева зависимостей... Готово
Чтение информации о состоянии... Готово
Уже установлен пакет nftables самой новой версии (1.0.9-1build1).
nftables помечен как установленный вручную.
Следующие пакеты устанавливались автоматически и больше не требуются:
  linux-headers-6.8.0-52 linux-headers-6.8.0-52-generic linux-image-6.8.0-52-generic
  linux-modules-6.8.0-52-generic linux-modules-extra-6.8.0-52-generic linux-tools-6.8.0-52
  linux-tools-6.8.0-52-generic
Для их удаления используйте «sudo apt autoremove».
Обновлено 0 пакетов, установлено 0 новых пакетов, для удаления отмечено 0 пакетов, и 227 пакетов не обновлено.
eavernikovskaya@ubuntu-katerok:~$
```

Рис. 26: Установка nftables

В некоторых дистрибутивах nftables уже установлен.

Установить его можно командой *sudo apt-get install nftables*

Практическое применение. Пример работы с nftables

Текущий набор правил
можно узнать
командой: *sudo nft list
ruleset*

```
eavernikovskaya@ubuntu-katerok:~$ sudo nft list ruleset
table ip filter {
    chain INPUT {
        type filter hook input priority filter; policy accept;
    }

    chain FORWARD {
        type filter hook forward priority filter; policy drop;
    }

    chain OUTPUT {
        type filter hook output priority filter; policy accept;
    }
}
```

Рис. 27: Текущие правила

Практическое применение. Пример работы с nftables

```
eavernikovskaya@ubuntu-katerok:~$ sudo nft add table inet test_table
eavernikovskaya@ubuntu-katerok:~$ sudo nft list ruleset
table ip filter {
    chain INPUT {
        type filter hook input priority filter; policy accept;
    }

    chain FORWARD {
        type filter hook forward priority filter; policy drop;
    }

    chain OUTPUT {
        type filter hook output priority filter; policy accept;
    }
}
table inet test_table {
}
eavernikovskaya@ubuntu-katerok:~$
```

Рис. 28: Создание таблицы (nftables)

При создании таблицы (table) должно быть определено семейство (family) адресов. Например, создадим таблицу с именем, test_table, которая обрабатывает одновременно пакеты IPv4 и IPv6: *sudo nft add table inet test_table*

Практическое применение. Пример работы с nftables

Для добавления цепочки
используется команда с таким
синтаксисом: *sudo nft add chain*
inet [таблица] [цепочка] {set}
Например создадим такую
цепочку: *sudo nft add chain inet*
test_table test_chain {type filter
hook input priority 0 ; policy
accept ; }. Эта цепочка
фильтрует входящие пакеты.

```
severnikovskaya@ubuntu-katerok:~$ sudo nft add chain inet test_table test_chain {type filter hook input priority 0 \
; policy accept ; }
severnikovskaya@ubuntu-katerok:~$ sudo nft list ruleset
table ip filter {
    chain INPUT {
        type filter hook input priority filter; policy accept;
    }

    chain FORWARD {
        type filter hook forward priority filter; policy drop;
    }

    chain OUTPUT {
        type filter hook output priority filter; policy accept;
    }
}
table inet test_table {
    chain test_chain {
        type filter hook input priority filter; policy accept;
    }
}
severnikovskaya@ubuntu-katerok:~$
```

Рис. 29: Создание цепочки (nftables)

Практическое применение. Пример работы с nftables

```
eavernikovskaya@ubuntu-katerok:~$ sudo nft add rule inet test_table test_chain ip saddr 8.8.8.8 drop
eavernikovskaya@ubuntu-katerok:~$ sudo nft list ruleset
table ip filter {
    chain INPUT {
        type filter hook input priority filter; policy accept;
    }

    chain FORWARD {
        type filter hook forward priority filter; policy drop;
    }

    chain OUTPUT {
        type filter hook output priority filter; policy accept;
    }
}
table inet test_table {
    chain test_chain {
        type filter hook input priority filter; policy accept;
        ip saddr 8.8.8.8 drop
    }
}
eavernikovskaya@ubuntu-katerok:~$
```

Рис. 30: Добавление правила (nftables)

Правила добавляются командой с таким синтаксисом: *sudo nft add rule [family] [table] [chain] [expression] [action]*

Теперь добавим правило: *sudo nft add rule inet test_table test_chain ip saddr 8.8.8.8 drop*.

Данное правило отбрасывает пакеты с ip-адресом источника отправления 8.8.8.8.

Практическое применение. Пример работы с nftables

Для удаления правила nftables используется команда со следующим синтаксисом: *sudo nft delete rule [family] [table] [chain] handle [number]*
Например *sudo nft delete rule inet test_table test_chain handle 2* удалит правило созданное нами на предыдущем шаге

```
eavernikovskaya@ubuntu-katerok:~$ sudo nft delete rule inet test_table test_chain handle 2
eavernikovskaya@ubuntu-katerok:~$ sudo nft list ruleset
table ip filter {
    chain INPUT {
        type filter hook input priority filter; policy accept;
    }

    chain FORWARD {
        type filter hook forward priority filter; policy drop;
    }

    chain OUTPUT {
        type filter hook output priority filter; policy accept;
    }
}
table inet test_table {
    chain test_chain {
        type filter hook input priority filter; policy accept;
    }
}
eavernikovskaya@ubuntu-katerok:~$
```

Рис. 31: Удаление правила (nftables)

Практическое применение. Пример работы с nftables

Далее для сохранения текущих правил iptables используем: *sudo nft list ruleset | sudo tee /etc/nftables.conf*

```
eavernikovskaya@ubuntu-katerok:~$ sudo nft list ruleset | sudo tee /etc/nftables.conf
table inet filter {
    chain input {
        type filter hook input priority filter; policy accept;
    }

    chain forward {
        type filter hook forward priority filter; policy accept;
    }

    chain output {
        type filter hook output priority filter; policy accept;
    }
}
table inet test_table {
    chain test_chain {
        type filter hook input priority filter; policy accept;
    }
}
```

Рис. 32: Сохранение текущих правил (nftables)

Далее после перезагрузки системы загрузим наши сохранённые правила: *sudo nft -f /etc/nftables.conf*

```
eavernikovskaya@ubuntu-katerok:~$ sudo nft list ruleset
[sudo] password for eavernikovskaya:
table ip filter {
    chain INPUT {
        type filter hook input priority filter; policy accept;
    }

    chain FORWARD {
        type filter hook forward priority filter; policy drop;
    }

    chain OUTPUT {
        type filter hook output priority filter; policy accept;
    }
}
eavernikovskaya@ubuntu-katerok:~$ sudo nft -f /etc/nftables.conf
eavernikovskaya@ubuntu-katerok:~$ sudo nft list ruleset
table ip filter {
    chain INPUT {
        type filter hook input priority filter; policy accept;
    }

    chain FORWARD {
        type filter hook forward priority filter; policy drop;
    }

    chain OUTPUT {
        type filter hook output priority filter; policy accept;
    }
}
table inet test_table {
    chain test_chain {
        type filter hook input priority filter; policy accept;
    }
}
eavernikovskaya@ubuntu-katerok:~$
```

Рис. 33: Загрузка конфигураций (nftables)

Практическое применение. Пример работы с nftables

Цепочка удаляется с помощью команды: *sudo nft delete chain [family] [table] [chain]*

Удалим цепочку test_chain: *sudo nft delete chain inet test_table test_chain*

```
eavernikovskaya@ubuntu-katerok:~$ sudo nft delete chain inet test_table test_chain
eavernikovskaya@ubuntu-katerok:~$ sudo nft list ruleset
table ip filter {
    chain INPUT {
        type filter hook input priority filter; policy accept;
    }

    chain FORWARD {
        type filter hook forward priority filter; policy drop;
    }

    chain OUTPUT {
        type filter hook output priority filter; policy accept;
    }
}
table inet test_table {
}
eavernikovskaya@ubuntu-katerok:~$
```

Рис. 34: Удаление цепочки (nftables)

Таблицу можно удалить конструкцией с синтаксисом: *sudo nft delete table [family] [table]*

Удалим таблицу test_table: *sudo nft delete table inet test_table*

```
eavernikovskaya@ubuntu-katerok:~$ sudo nft delete table inet test_table
eavernikovskaya@ubuntu-katerok:~$ sudo nft list ruleset
table ip filter {
    chain INPUT {
        type filter hook input priority filter; policy accept;
    }

    chain FORWARD {
        type filter hook forward priority filter; policy drop;
    }

    chain OUTPUT {
        type filter hook output priority filter; policy accept;
    }
}
eavernikovskaya@ubuntu-katerok:~$
```

Рис. 35: Удаление таблицы (nftables)

Сравнение iptables, nftables и eBPF

Метод	Где работает?	Плюсы	Минусы
iptables	Пользовательское пространство	Прост в использовании	Низкая производительность, неэффективен при большом количестве правил
nftables	Пользовательское пространство	Гибкость, выше скорость, чем у iptables	Требует новых знаний
eBPF XDP	Драйвер сетевой карты	Максимальная скорость, подходит для DDoS-защиты	Ограниченные возможности обработки пакетов
eBPF TC	Сетевой стек ядра	Гибкость, возможность модификации пакетов	Чуть сложнее, чем XDP
eBPF SOCK FILTER	Уровень приложений	Позволяет фильтровать на уровне сокетов	Не применимо для всей сети

Рис. 36: Таблица. Сравнение iptables, nftables и eBPF)

Вывод

Пакетный фильтр на базе Linux является мощным и гибким инструментом, который позволяет эффективно управлять сетевым трафиком и обеспечивать защиту от киберугроз. Современные технологии, такие как nftables и eBPF, делают фильтрацию более эффективной, а мониторинг трафика с помощью инструментов логирования позволяет оперативно реагировать на потенциальные угрозы. Использование современных средств фильтрации в Linux позволяет создать надежную систему защиты и поддерживать высокий уровень безопасности в сетевых инфраструктурах!



Список литературы

1. ProstoKirReal. Сложно о простом. Самые популярные заголовки уровня L2 модели OSI в Ethernet. Habr, 2024. [Электронный ресурс] URL: <https://habr.com/ru/companies/timeweb/articles/836038/>
2. ProstoKirReal. Сложно о простом. Самые популярные заголовки уровня L3 и L4 модели OSI. Habr, 2024. [Электронный ресурс] URL: <https://habr.com/ru/companies/timeweb/articles/846878/>
3. Настольная книга администратора Debian. Debian. [Электронный ресурс] URL: <https://1.github.io/debian-handbook/html/ru-RU/sect.firewall-packet-filtering.html>
4. Трасковский А. Настройка Iptables для чайников. timeweb, 2021. [Электронный ресурс] URL: <https://timeweb.com/ru/community/articles/nastroyka-iptables-dlya-chaynikov>

